Cloud Security Posture Monitor

Automated AWS Misconfiguration Detection Tool

Krithika | MSc Data Science, AIMIT St. Aloysius (Deemed to be University), Mangaluru

1. Abstract

Cloud platforms such as Amazon Web Services (AWS) allow organisations to store data and run applications without owning hardware. Many real-world data breaches, however, do not come from advanced attacks. They come from simple misconfigurations, such as a storage bucket left public or a firewall rule left open to the whole internet. This project presents the Cloud Security Posture Monitor, a Python tool that connects to an AWS account through a read-only IAM user, checks S3 buckets, IAM users and roles, and security groups against security rules, rates each finding as HIGH or MEDIUM, and produces a terminal summary and an HTML report. The tool was tested on a real AWS account and correctly detected a risky security group that was created on purpose.

2. Introduction and Problem Statement

Moving to the cloud shifts part of the security responsibility to the customer. AWS secures the infrastructure, but the customer must configure storage, permissions and firewalls correctly. A single forgotten setting can expose private data. Tools that continuously check cloud settings against best practices belong to a category called Cloud Security Posture Management (CSPM).

Problem statement: Manually reviewing every storage bucket, user permission and firewall rule in a cloud account is slow and error-prone. There is a need for a simple automated tool that finds common misconfigurations and reports them clearly.

3. Objectives

- Build a Python tool that scans an AWS account for common security misconfigurations.
- Check three areas: S3 buckets, IAM users and roles, and security groups.
- Rate each finding by severity (HIGH or MEDIUM).
- Generate a readable terminal summary and an HTML report.
- Follow security best practices: least privilege and safe handling of access keys.

- Run the tool locally on a personal computer.

4. Scope and Limitations

Scope: The tool audits only the AWS account that owns the access keys. It is a self-audit tool and does not scan other people's accounts.

Limitations:

- It scans one region (ap-south-1, Mumbai).
- It reads only policies attached directly to users and roles, not inline policies or group permissions.
- It has three checks. Professional tools such as Prowler and AWS Security Hub cover hundreds.
- It reports problems but does not fix them.

5. Technologies Used

Component	Tool	Purpose
Language	Python 3.10	Scanner logic and report generation
AWS SDK	Boto3	Talking to AWS services through their APIs
Cloud services	S3, IAM, EC2 (security groups)	The resources being audited
Access control	IAM user and custom policy	Read-only access for the scanner
Editor and terminal	VS Code, Windows PowerShell	Writing and running the code
Report	HTML and CSS	Colour-coded findings report

6. System Design

Flow of the tool:

1. The scanner connects to AWS using the read-only IAM user.

2. It requests the current settings of S3 buckets, IAM users and roles, and security groups.
3. Each resource is tested against security rules.
4. Every result is stored in one list with the resource type, name and any findings.
5. The list is printed as a terminal summary and written into report.html.

Severity levels:

Level	Meaning	Colour in report
PASS	No issue found	Green
MEDIUM	A safety setting is off, but nothing is exposed yet	Orange
HIGH	The resource is actually exposed or too powerful	Red

7. Security Design

- **Dedicated IAM user:** A separate user named security-posture-scanner was created with programmatic access only and no console login.
- **Least privilege:** A custom policy named security-scanner-readonly-policy allows only read actions. For S3 it allows listing buckets and reading bucket ACLs, policies, policy status and public access block settings. For IAM it allows listing users, roles and attached policies and reading policy documents. For EC2 it allows describing security groups. The tool cannot create, change or delete anything.
- **Credential handling:** Access keys are supplied through environment variables and are never written in the code or uploaded to GitHub.
- **Key lifecycle:** Keys are created for use, then deactivated and deleted afterwards. One key was displayed in a screenshot during testing, so it was deleted immediately and replaced with a new one.

8. Implementation

8.1 Connection test

A small script (test_connection.py) lists the S3 buckets in the account. It confirms that the keys, region and Python setup work before any check is written.

8.2 Check 1: S3 buckets

For every bucket the tool applies three rules:

Rule	Severity if failed
Block Public Access is not fully enabled	MEDIUM
The bucket policy makes the bucket public	HIGH
The ACL grants access to everyone (AllUsers or AuthenticatedUsers)	HIGH

8.3 Check 2: IAM users and roles

The tool lists every IAM user and every custom role, reads the policies attached to them, and flags any policy that allows all actions on all resources (full administrator access) as HIGH. AWS service-linked roles are skipped because AWS manages them.

8.4 Check 3: Security groups

The tool reads every inbound rule and flags rules open to 0.0.0.0/0 (IPv4) or ::/0 (IPv6):

Situation	Severity
All traffic open to the internet	HIGH
A sensitive port is open: SSH (22), RDP (3389), MySQL (3306), PostgreSQL (5432), MS SQL (1433), MongoDB (27017), Redis (6379)	HIGH
ICMP (ping) open to the internet	MEDIUM
Other ports open, except web ports 80 and 443	MEDIUM

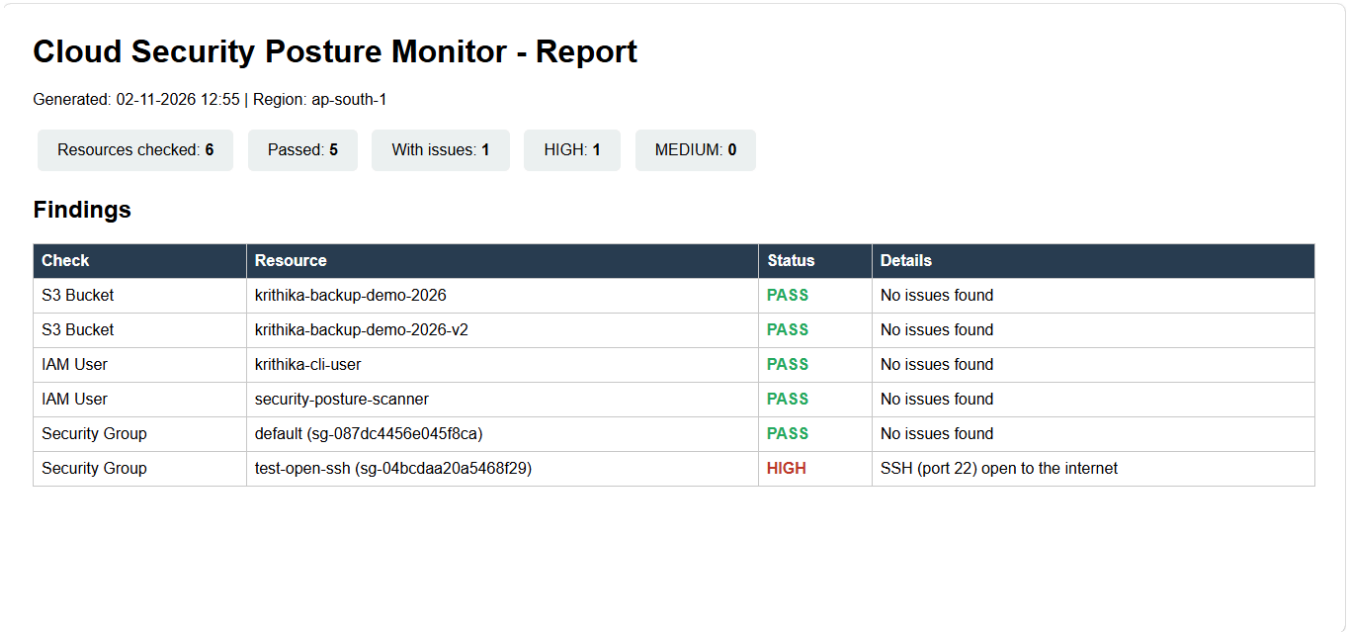
8.5 Combined scanner and report

The three checks were merged into a single file, scanner.py. It prints one line per resource ([OK] or [!] with details), a summary line with totals, and saves report.html. The report shows summary boxes (resources checked, passed, with issues, HIGH, MEDIUM) and a table of every resource with colour-coded status.

9. Testing and Results

Test	Resources checked	Passed	Findings
Normal account (2 S3 buckets, 2 IAM users, 1 default security group)	5	5	None
Test security group with SSH (port 22) open to 0.0.0.0/0 added	6	5	1 HIGH: SSH open to the internet
Test security group deleted, scan repeated	5	5	None

Observation: The scanner flagged the risky rule when it existed and returned to a clean result after the rule was removed. This shows that it detects real problems without raising false alarms on safe resources. The test security group was not attached to any server, so nothing was actually exposed during testing.



Cloud Security Posture Monitor - Report

Generated: 02-11-2026 12:55 | Region: ap-south-1

Resources checked: **6** | Passed: **5** | With issues: **1** | HIGH: **1** | MEDIUM: **0**

Findings

Check	Resource	Status	Details
S3 Bucket	krithika-backup-demo-2026	PASS	No issues found
S3 Bucket	krithika-backup-demo-2026-v2	PASS	No issues found
IAM User	krithika-cli-user	PASS	No issues found
IAM User	security-posture-scanner	PASS	No issues found
Security Group	default (sg-087dc4456e045f8ca)	PASS	No issues found
Security Group	test-open-ssh (sg-04bcdaa20a5468f29)	HIGH	SSH (port 22) open to the internet

Figure 1: Report generated on 02-11-2026. The test security group is flagged HIGH because SSH (port 22) is open to the internet, while the other 5 resources pass.

10. Challenges and Solutions

Challenge	Solution
Two Python versions were installed and pip was blocked in one of them	Installed Boto3 in Python 3.10 and ran every script with py -3.10
InvalidAccessKeyId errors	Found that keys were deleted or pasted incorrectly, checked key lengths (20 and 40 characters) and created a fresh key
An access key appeared in a screenshot	Deleted and replaced the key, then cleared the screen with cls after setting keys
A ping (ICMP) rule produced a confusing message about ports -1 to -1	Added explicit handling so it reports ICMP (ping) open to the internet

11. Ethical Considerations

The tool is defensive. It uses read-only access and audits only the owner's own account. Scanning an account without permission is unauthorised access, even when only reading settings, so the tool is not designed or intended for that.

12. Future Scope

- Check that IAM users have multi-factor authentication (MFA) enabled.
- Flag access keys older than 90 days.
- Verify that CloudTrail logging and S3 encryption are enabled.
- Scan multiple regions and inline or group-based policies.
- Run automatically using AWS Lambda and EventBridge on a schedule.
- Send email alerts through Amazon SNS when a HIGH finding appears.
- Add a web dashboard to view results over time.

13. Conclusion

The Cloud Security Posture Monitor shows how common cloud misconfigurations can be found automatically with a small Python program. It combines cloud computing (S3, IAM, EC2 and API-based access) with cyber security practices (least privilege, exposure

detection, severity rating and safe key handling). Testing confirmed that the tool reports real risks accurately and that it is a practical learning version of the tools used in industry.

14. How to Run the Project

1. Create or activate an access key for the security-posture-scanner IAM user.
2. Open the project folder in VS Code and open the terminal.
3. Set the environment variables AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY and AWS_DEFAULT_REGION (ap-south-1), then clear the screen.
4. Run: py -3.10 scanner.py
5. Open the report: start report.html
6. After use, deactivate and delete the access key.

15. References

- AWS documentation: Identity and Access Management (IAM), Amazon S3 and Amazon EC2 security groups.
- Boto3 documentation (AWS SDK for Python).
- AWS Well-Architected Framework, Security Pillar.